K215: automatic source certificates, concrete credit proofs, and the 214-state search

A research and integration report for ProveIt

21 October 2026

Abstract

The current 215-state machine already has checked Lean and Rocq correctness endpoints. The useful next step is therefore to preserve that calibration and remove repeated source-correspondence and enumeration arguments. This report reviews all seven pending proposal archives, supplies a new Rocq-compatible normalized source checker with an exact all-register theorem, implements automatic correspondence inference in Lean from the two literal decoder trees, and proves the concrete zero-credit scheduler theorem for the existing ZC215 codec. The first construction reduces the source relation from 2,240 entries to 1,426. The second removes a separately maintained correspondence input by synthesizing a cache and checking it against the source trees. The third discharges the actual codec, parser, monotonicity, and injectivity assumptions behind the previous abstract credit argument. A separate predecessor-specific state elimination search and a targeted scanner repair search did not construct a verified 214-state machine. Their finite scope is recorded explicitly.

1 Results, scope, and recommendation

The reviewed repository is pinned to commit `53269bcab1efc0f82d3022645bd7cf0c22d84959`. Unless stated otherwise, paths below are relative to `Computability/BusyBeaver/Research/ZFC/`. All seven ZIP archives then present in `docs/incoming/` were inspected; their filenames, hashes, and individual contributions are recorded in the accompanying incoming-proposal inventory. The newest archive is the one whose filename ends in `(2).zip`. The main candidate and the established endpoints are inherited work, not new results of this report.[1, 3]

Component	Contribution here	Exact boundary
Normalized Rocq source proof	1,426-entry checker and all-register equivalence for literal `p215` and `dagPerm`	Coq 8.18.0 build; full binary/ZFC closure and Rocq 9.1 integration are separate
Automatic Lean certificate	Infer and cache paths and successors from the two decoder trees; validate with the proved normalized checker	Source equivalence; no machine-state reduction
Concrete ZC215 credit	Exact Cantor codec, parser inverses, lowering injection, and all-prefix credit in one closed development	Scheduler theorem; not a new ZC215 binary endpoint
State reduction	Predecessor-dependent elimination across 21 released tables, plus scanner repair	No accepted 214-state construction; no lower bound

The practical recommendation has three parts. First, retain the already certified K215 as the primary endpoint and adopt proof refactors behind the same public statements. Second, use the

existing conditional-marker CM215 when the aim is the shortest additional formal correctness proof: the incoming packets already contain its exact relative halting bridge. Third, use ZC215 when simplifying the enumerator's mathematical explanation is the priority; its tag-lowering injection gives an especially direct protected-stack argument. These are different measures of simplicity. A short proof relative to K215 still inherits K215's foundational and compiler development.

214-state answer. No verified 214-state or smaller machine was constructed in this investigation. No rejected search output is offered as a candidate satisfying the consistency specification. The useful positive deliverables are the proof components and the reproducible search framework.

1.1 Meaning of the target

The desired statement is

$$\mathsf{ZFC} \vdash (\neg\,\mathrm{Halt}(M, \mathrm{blank}) \iff \mathrm{Con}(\mathsf{ZFC})),$$

where consistency is syntactic nonexistence of a finite refutation in the specified proof system. This is an equivalence theorem, not an unconditional claim that either side holds. Its elementary machine and source transformations can be formalized in ZFC. A checked external Lean or Rocq theorem about the corresponding predicates must also be distinguished from a proof object encoded inside a formalization of ZFC or PA itself. The repository documents an internal PA endpoint for the earlier 240-state machine; the current K215 external endpoint does not automatically settle that separate milestone. [1]

All machine counts use the conventional binary, one-tape, two-way infinite tape model with L/R moves, blank input, and initialization included. The separate halt is excluded from the number of working states. A 215-state table therefore has 430 working transition rows. Merely using a large ambient type for labels does not add reachable working states; conversely, renaming states does not remove one.

2 What the existing proposals already establish

The current repository contains substantial simplification work. Its phase-four record reports a 185-module Lean closure with ten freshly rebuilt affected modules and 175 precisely bound reused objects. It shares implication lemmas, removes a redundant segment-halting induction, and proves enumerator blocks through symbolic execution. The three highlighted files decrease from 1,814 to 1,606 lines. Those are existing improvements and are not credited to this packet. The underlying full endpoint has a separate fresh-build history. [2]

The incoming proposals also overlap considerably. A review that reads only the main candidate's README misses several already-compiled follow-up results. The following inventory records the main distinctions; each entry is an archive-level claim, not an assertion that its entire inherited endpoint was freshly rebuilt during this work.

Archive stem, all dated 2026-10-21	Already supplied contribution
`K215_Proof_Simplification (2)`	Exact CM215 dense/Core bridge; ordinary zero-offset source equivalence; Coq certificate identity and mirror checker; a final full-ZFC adapter with an explicit integration boundary.

Archive stem, all dated 2026-10-21	Already supplied contribution
`K215_Proof_Simplification`	Compiled 1,426-entry normalized Lean source equivalence; compiled CM215 transfer; generic positive stuttering.
`K215_Proof_Transport`	Exact CM215 Core bridge, positive-block theorems, source-graph measurements, and bounded reduction searches.
`Structural_Proof_Simplification`	Equal canonical weighted test graphs established by finite analysis; structural and semantic proposals. Its canonical certificate is not accepted by the unchanged raw-path checker.
`ZFC215_Proof_Simplification`	Plain one-sided Coq source proof, compact paths, and a marker-variant development with stated adapter limits.
`ZFC_K215_Proof_Transport`	Standalone all-tape CM215 proof, shared-tree and compact-path pilots, with the remaining representation adapter identified.
`zfc-fm215-proof-compression`	Four-row FM215 variant, standalone all-tape equivalence, decoder-context compression, and a restricted tail-quantifier countermodel.

Two consequences guide the present work. Recommending mirror symmetry, zero-offset specialization, or shared trees alone would repeat existing proposals. More substantially, the 1,426-entry Lean certificate is already checked, so its case-count reduction is not a new discovery here. The new work is to bring the Rocq lane to that reduced relation and to remove an explicit correspondence file from the Lean-facing interface. The concrete scheduler proof similarly formalizes an existing mathematical idea while closing hypotheses that the old abstract proof left to its caller.

3 Normalized source correspondence in the Rocq lane

3.1 Why 814 entries can be removed

The path machine reads a finite Boolean pointer as if it were followed by infinitely many zero bits. Define

$$P \simeq Q \quad \Longleftrightarrow \quad \forall i \in \mathbb{N},\ \mathrm{bit}(P, i) = \mathrm{bit}(Q, i),$$

where absent bits are zero. For example, $(1, 0) \simeq (1, 0, 0, 0)$. After it finds a primitive leaf, the decoder executes using the actual consumed path. Padding is therefore a representational artifact, not an observable difference in the register computation.

The old certificate has 2,240 path pairs. Normalizing the trailing-zero aliases leaves 1,426 relevant pairs, a reduction of

$$2240 - 1426 = 814, \qquad \frac{814}{2240} \approx 36.34\%.$$

There are 1,425 decrement checkpoints and one halt checkpoint. Both outcomes of every decrement still require validation, as does the initial segment. The reduction does not throw away a branch because it was absent from a bounded execution trace.

Lemma 3.1 (Path aliases preserve halting)**.** *For any source program p, register file R, and pointers $P \simeq Q$, the executions from (P, R) and (Q, R) have the same halting predicate. The finite halting-witness transport is already available in the inherited path semantics.*

Proof. The decoder observes identical bits at every position, so it selects the same primitive instruction and consumed depth. The zero-extended prefixes of that depth agree. Consequently

the next register update, next pointer, and literal halt observation agree. Transport a finite run by induction. The formal development reuses `RMEquiv.hn_ipeq`; it does not introduce a second path semantics or reinterpret a failed decoder as successful halting. □

The executable Boolean comparison `ibeq` checks this relation without constructing an infinite bit sequence. It compares common prefixes and accepts an unmatched suffix exactly when all its bits are false. Its soundness theorem is closed under the global context.

3.2 Segments and the reduced checker

Between decrement tests, the source executes only increments and structural break/continue instructions. Such a segment does not inspect a register. It has a finite increment-count vector and a final control pointer. The inherited evaluator `seg` computes that information with a fuel bound; fuel exhaustion is rejected.

For a certificate entry (P_1, P_2, j, k), the new checker verifies either that both leaves are halt, or that both decrement the same register and:

1. the nonzero branches have equal increment counts and end at pointers equivalent under $\simeq$ to the pair indexed by j;
2. the zero branches satisfy the corresponding condition for k.

The existing source programs, segment evaluator, register updates, and halting predicate remain the inputs to this test. Only the endpoint comparison in the certificate checker changes from literal finite-list equality to $\simeq$.

Theorem 3.2 (One checked normalized certificate suffices)**.** *If the initial segment condition and every normalized entry condition succeed, then for every register file R,*

$$\mathrm{rm_halts}(p_1, ([], R)) \iff \mathrm{rm_halts}(p_2, ([], R)).$$

Proof. For the forward implication, induct on the length of a finite halting run. A related halt pair ends the argument. At a decrement pair, equal register files choose the same branch. That decrement consumes a source step, so the induction parameter strictly decreases. The two following segments produce the same register increments. Transport their final pointers to the indexed certificate pointers by the alias lemma, and apply the induction hypothesis. The initial segment is handled in the same way, without needing it to have positive length.

For the reverse implication, swap the two pointer components of every entry. The increment-count equality is symmetric, and the two endpoint alias checks exchange positions. A generic mirror theorem establishes acceptance of the mirrored certificate from the already-proved forward acceptance. Apply the same soundness theorem. No second large computed certificate check is needed. □

3.3 Concrete checked theorem and costs

The delivered `Source215Normalized.v` checks the literal inherited `Prog215.v` and `DagPerm.v` and proves:

```
Theorem p215_dagPerm_normalized R :
  rm_halts p215 ([], R) <-> rm_halts dagPerm ([], R).
```

The Boolean certificate theorem and this endpoint print closed assumption reports. All twelve selected project modules compiled freshly, and ordinary recursive `coqchk` passed without `-admit` or `-norec`.

The active data source is 109,115 bytes. The incoming ordinary 2,240-entry counterpart is 171,678 bytes, so the like-for-like data saving is 62,563 bytes, approximately 36.44%. This comparison should not be confused with the much larger saving obtained earlier by replacing expanded Boolean path lists with binary-coded paths. The new generic checker is an additional reusable proof component and must be counted when measuring the total development.

The new reusable checker is 196 lines (8,032 bytes), and the concrete source instance is 24 lines (901 bytes). Relative to production's two separately computed 2,240-entry directions, concrete entry checks decrease from 4,480 to 1,426. The mirror technique already appeared in the earlier one-sided proposal; only the normalization-aware Rocq proof and instance are new here.

All twelve compiler children took 65.99 seconds in total, with peak resident memory 816,160 KiB. The concrete source module took 15.35 seconds; its forward `vm_compute` transaction reported 7.006 seconds. Recursive `coqchk` took 543.73 seconds and peaked at 338,332 KiB. All five new printed assumption reports were closed. These are local measurements of the selected source closure, not a full-project speedup comparison.

The reproducible build uses Coq 8.18.0. A small, proved compatibility layer provides renamed standard-library facts required by the imported Rocq-era source; the original source and the exact build transformation are retained. This is not a claim that the complete 237-source production endpoint has been rebuilt under Rocq 9.1.0. The supplied integration wrapper preserves the old source theorem's public role, but production adoption should be followed by that repository's normal full endpoint and assumption audit.

4 Automatic Lean correspondence inference

4.1 Removing a separately maintained input

The normalized Lean proposal still ships an explicit 1,426-entry table of paired pointers and successor indices. The structural proposal observes that the two trees' test leaves correspond by their order. We turn that observation into an executable certificate generator and retain the existing proved checker as the acceptance boundary.

The generator takes only the two decoder trees. It performs the following finite operations:

1. Traverse each tree from left to right and list the full consumed paths of its decrement and halt leaves.
2. Pair these lists in order. Encode each path by a leading sentinel one followed by its bits.
3. For each source decrement, compute its zero and nonzero continuation segments in the first tree. Determine the resulting checkpoint's rank in that tree. Use those ranks as the proposed successor indices.

4. Build a balanced certificate and run the normalized segment checker against *both* actual trees.

The rank computation follows the pointer down the decoder tree, adding the number of checkpoints in a left subtree whenever it takes a right edge. It therefore avoids a linear search through every checkpoint path. As in the source semantics, exhausted pointer lists are interpreted with trailing zeros.

The generator is deliberately a heuristic rather than a theorem that every equivalent pair of source programs has this particular ordering. A pair of equivalent programs with a different checkpoint order can be rejected. That incompleteness is harmless: acceptance is established by the already-proved checker, not by an assumed correspondence property of the generator.

Theorem 4.1 (Sound automatic inference)**.** *Let* $C = \text{synthesize}(T_1, T_2)$*. If the normalized initial and entry checks succeed for* C*, and* T_i *are the actual decoder trees of* p_i*, then the two source programs have equivalent halting predicates from the empty pointer for every register file.*

Proof. Instantiate the normalized checker's soundness theorem with C. Nothing about how C was obtained is required. The proof is valid even if a generator branch uses a fallback index, construction fuel is insufficient, or the two path lists have different lengths: those situations must still survive every semantic acceptance condition. In particular, an empty generated relation does not pass the required initial relation check. The theorem consequently adds no trusted Python computation, hash-based equality, or generator-correctness axiom. □

This formulation is smaller than a full verified compiler from the source language to a new graph semantics. It uses the structural proposal's insight without requiring a second operational semantics and adequacy proof. It also has a precise limit: it removes the *manually maintained* correspondence input, not the mathematical relation or the checking work. The executable generator computes the relation, emits literal data, and submits that data to the established kernel checker.

4.2 Actual K215 instantiation

The generic inference module is 88 lines. A small executable Lean emitter computes the relation from the existing exact shared trees and partitions it into sixteen literal checking blocks. The emitted data is a reproducible cache, not a trusted proof. The final theorem uses the same public all-register target:

```
theorem source_halts_iff (R : Registers) :
  Halts p215 <[], R> <-> Halts pathDagPerm <[], R>
```

Here the angle brackets in the display stand for Lean's configuration constructor. The delivered file contains the actual Lean syntax.

The acceptance computations are expressed through `decide +kernel`. The development does not use `native_decide`, an admitted theorem, or a finite-run assertion in place of the all-register argument. An initial executable evaluation independently returned 1,426 rows, a successful initial check, and successful checks for all entries. Executable evaluation alone is not presented as a proof of the concrete result. The final source theorem passed kernel checking. All thirty selected project modules were compiled from source in the isolated build; successful modules were hash-bound across the documented resource retries. The final printed axioms are exactly `propext`, `Classical.choice`, and `Quot.sound`.

The thirty accepted compiler invocations took 578.41 seconds in total; the successful process records, including emitter replays, sum to 588.20 seconds. Peak accepted resident memory was 3,954,196 KiB. Two staged-check resource failures consumed a separately recorded 162.09 seconds. These are sums of recorded process times across the isolated development build, not the wall time of a single uninterrupted clean command. The final cache regenerates byte for byte; the supplied default fresh-build driver uses the accepted final sources and a 4,000 MiB cap.

The current repository's `RMBisim.lean`, including its phase-four successful-prefix cancellation lemma, is used in the fresh build. The eight inherited source-semantics/DAG dependencies are bound to the current commit by Git blob identity. The source programs' literal node table and decoder-tree bindings are also audited. Shared trees and the normalized checker are credited to the incoming proposal from which they are reused.

4.3 Engineering tradeoff

The first implementation made each checking block reduce the inferred relation definition directly. Its executable evaluation accepted all 1,426 entries, and the first three blocks passed kernel checking, but block four exceeded the configured 3,000 MiB memory limit. That failure is retained in the experimental receipt; it is not a mathematical rejection of the certificate.

The delivered production-style experiment therefore stages the computation. Lean executes the maintained inference algorithm once and writes a literal balanced certificate. The kernel then validates every entry against the two actual source trees. There is deliberately no theorem equating that cache with the generator's output: such an equality is unnecessary for semantic soundness and would repeat the expensive computation. The exact source-equivalence theorem depends on checker acceptance, irrespective of who generated the data. Later checking blocks also use the incoming proposal's quarter-block batching to bound kernel memory. Resource retries are recorded separately from accepted compilations. The build receipt records reproducibility of the delivered cache, which is useful evidence about engineering identity but is not a logical premise.

A smaller source file is not automatically a faster proof. Synthesis performs additional tree traversals, and evaluating a definition again in multiple checking blocks can be expensive. The recorded times are measurements of this build, not a benchmark against a fresh complete baseline under identical load. The report makes no speedup claim without such a comparison.

4.4 One inferred correspondence for both assistants

The same newly inferred 1,426 tuples also pass the Rocq normalized checker. The additional `rocq_auto/` component renders the emitter's literal `entries.json` through the existing sentinel-bit path codec and proves the exact all-register `p215/dagPerm` theorem again. This is a useful cross-assistant interface: maintain one inference algorithm, then let the independently established semantic checkers validate its output against each assistant's literal programs. Neither checker assumes the correctness of the generator or of the other assistant.

The two new Coq modules compiled in 11.798 and 19.429 seconds; the acceptance transaction reported 7.378 seconds. All three printed assumption reports are closed. This additional check imports ten source-and-object-hash-bound dependencies from the already accepted normalized build. It is an incremental two-module validation, not another fresh dependency build or recursive `coqchk`

run. The first normalized source theorem retains the separate successful recursive audit reported above.

The automatically inferred Coq data is 110,668 bytes, slightly larger than the earlier 109,115-byte normalized data because it retains full consumed paths. The benefit is a single maintained inference procedure and two semantic validations, not a claim that every generated representation is smaller.

5 A concrete, closed prefix-credit theorem for ZC215

5.1 Literal codec

ZC215 is an existing 215-state alternative. Its important source change is that its scheduler processes raw candidates $0, 1, 2, \ldots$, with DROP assigned tag zero and NIP tag one. The old mathematical tag-lowering proof is already in the repository. The older `PrefixCredit.v` draft abstracted the lowering map and its properties. Our new development proves those properties for the actual codec and a total executable parser.[5]

Write Cantor pairing in the source's coordinate convention as

$$\mathsf{P}(a, b) = \frac{(a+b)(a+b+1)}{2} + b.$$

The command fields are:

command	fields
DROP	$[0]$
NIP	$[1]$
MP(x)	$[2, x]$
AX$(c; x, y)$	$[c+3, y, x]$

Let $\mathrm{emit}(k, r)$ nest those fields through pairing, with final residual r. Define

$$\mathrm{code}([]) = 0, \qquad \mathrm{code}(k :: w) = 1 + \mathrm{emit}(k, \mathrm{code}(w)).$$

The raw candidate of a nonempty word is its code minus one. This final shift must be retained: raw candidate zero represents the actual command word [DROP], and raw candidate two represents [DROP, DROP].

Theorem 5.1 (Concrete parser inverses)**.** *The delivered executable parser* word *satisfies*

$$\mathrm{code}(\mathrm{word}(n)) = n + 1$$

for every n, and

$$\mathrm{word}(\mathrm{code}(w) - 1) = w$$

for every nonempty word w.

The formal proof constructs Cantor unpairing, proves the required inverse and coordinate inequalities, and recurses on a strictly smaller residual. Thus the parser and the code injection are proved, not supplied as abstract premises. The explicit residual convention prevents a common error: a final zero command tag must not be mistaken for an absent command.

5.2 Tag lowering is a well-founded injection

An occurrence is a pair (n, j) consisting of a raw candidate and a command position. For a NIP occurrence, replace that NIP with DROP in word(n), re-encode the resulting word as m, and set

$$L(n, j) = (m, j).$$

Lemma 5.2 (Three properties of lowering)**.** *The image is a DROP occurrence, $m < n$, and L is injective on NIP occurrences.*

Proof. The first assertion follows from the parser inverse: decoding the new word's code recovers that word, with DROP at the changed position. For strict decrease, pairing is strictly increasing in either coordinate when the other coordinate is fixed. Replacing tag one by tag zero therefore lowers the code at the chosen command. Every preceding command encoder is strictly increasing in its residual argument, so the decrease propagates to the entire raw code.

For injectivity, suppose two lowered occurrences are equal. Their candidate and position agree. Decode that candidate and restore NIP at the specified position. This recovers both original words. The code/parser inverse then recovers the same original raw candidate. Hence both components of the original occurrences agree. □

Notice that the image is in a *strictly earlier candidate*, not merely at an earlier position in the same candidate. This is what makes the argument work uniformly at every prefix, including prefixes that stop immediately before a NIP.

5.3 Every canonical enumeration prefix has credit

Let prefix(n, j) contain all occurrences in candidates $0, \ldots, n-1$, followed by positions $0, \ldots, j-1$ of candidate n. Let $D(n, j)$ and $N(n, j)$ be the numbers of DROP and NIP occurrences there.

Theorem 5.3 (Nonnegative credit and strict pre-pop credit)**.** *For all n, j,*

$$N(n, j) \le D(n, j).$$

If candidate n has a NIP at position j, then

$$N(n, j) + 1 \le D(n, j).$$

Proof. Apply the lowering injection to the NIP occurrences in the prefix. Each image belongs to an earlier completed candidate and is therefore among the prefix's DROP occurrences. The source and target occurrence lists have no duplicate occurrence identifiers, so injectivity gives the first cardinality inequality.

For the stronger result, include the current NIP occurrence (n, j) among the injection's sources even though it is not yet in the prefix. Its image still lies in an earlier completed candidate. It is distinct from the images of all earlier NIPs. There are therefore at least $N(n, j) + 1$ earlier DROPs. □

The formal statements have no assumptions about codec injectivity, monotonicity, surjectivity, a generic enumeration, or a distinctness invariant. Those facts are proved internally. The prefix theorem remains valid for j beyond the length of the current word, since invalid positions belong to neither filter.

5.4 Protected stack tail

The intended stack invariant is a logical witness of an explicit list of proved formula heads above an arbitrary opaque initial tail. If its available height h satisfies

$$D(n, j) \leq h + N(n, j),$$

then the strict credit theorem gives $h > 0$ immediately before a NIP. DROP provides one head, NIP consumes one, and the remaining command contracts are required not to reduce the conservative credit assigned here. AX may be assigned zero credit in this argument.

That last observation is a proof simplification, not authorization to remove AX outputs from a machine. Such a machine change could alter completeness or the numerical calculus and would need its own calibration. Similarly, the logical height is not a numerical size measure of the encoded stack. Since $\mathsf{P}(0, 0) = 0$, zero can encode multiple explicit truth heads above a zero tail. A witness list, rather than the natural number's magnitude, is the correct invariant.

`ConcretePrefixCredit.v` has been compiled with Coq 8.18.0 and recursively checked with `coqchk`. The two parser inverses, injection, two credit theorems, and protected-head corollary all print closed global-context reports. The delivered reconstruction script records exact source hashes, commands, compiler output, and kernel-check results. A separate finite parser comparison binds the conventions to the released source model; it supplements rather than replaces the unbounded formal proof.

That comparison extracts the actual proved Coq parser to OCaml and matches every command and field of candidates zero through 999. It also interprets the literal 4,752-leaf source: 4,000 isolated field-reader cases, 1,000 refills, 1,000 positive continuations, and 1,000 dispatch paths pass, using 19,686,389 source transitions. Complete AX/MP bodies are outside this check. The remaining unbounded parser/loop bridge is required before the canonical prefix theorem can be presented as a theorem about every literal machine prefix.

6 Which 215-state candidate is easier to prove?

6.1 Shortest additional proof: CM215

CM215 is the best current choice for a short *relative* correctness proof. Its six-row change from K215 admits a whole-tape simulation at identical head positions, with each old step represented by one or three new steps. The relation maps old control 176 to new control 52 and otherwise retains controls. Three simple local tape conditions distinguish the merged roles. Incoming proposals already contain a compiled bridge to the exact existing dense/Core machine semantics, so re-proving its numerical calculus, enumerator, compiler, and blank startup would duplicate established work.[4, 3]

The reusable principle is worth stating with all hypotheses visible. Here a terminal configuration means one whose partial successor function is undefined, and terminal status agrees in both directions at every related pair.

Proposition 6.1 (Positive-block transport)**.** *For deterministic partial transition systems, suppose an initial relation preserves terminal observations, preserves a source invariant, and maps each nonterminal source step to a strictly positive finite target block ending in the relation. Then the related initial states have equivalent halting predicates.*

Proof. Forward transfer concatenates the simulated blocks. For reverse transfer, induct on a finite target halting length. The source-terminal case is decided by terminal agreement. Otherwise the positive target block consumes at least one target step; determinism makes that block a prefix of any target halting run, so cancel it and apply induction at the related endpoints. The decrease would fail for zero-length blocks. This theorem is already checked in the incoming proposals; it is cited here as an integration tool, not as a new contribution. □

This does not make CM215's entire proof development independent of K215. It gives a short new layer over a shared certified base. The final ZFC adapter must be compiled in the established project before advertising a freshly rebuilt CM215 end-to-end endpoint. The incoming packets themselves distinguish that wrapper from their completed local bridge.

6.2 Simplest scheduler explanation: ZC215

ZC215 changes the scheduler and startup, retaining the command bodies of its command-permuted parent. Its clean raw enumeration and lowering injection make the stack-protection argument particularly transparent. The concrete credit theorem in this packet now supplies a checked implementation of that mathematical advantage. But the source-loop and binary-startup connections are additional integration work; the theorem alone is not the complete ZC215-to-ZFC endpoint.

The combined GC-ZC215 alternative makes further changes to generalization and decoder allocation. Those can be interesting for state reduction, but they introduce additional obligations if proof simplicity alone is the objective. The four-row FM215 alternative is also attractive for a compact local proof, and its incoming packet already supplies one. Its remaining repository semantics adapter should be counted when comparing current formal readiness.

Candidate	Reason to choose it	Work still relevant
Primary K215	Existing full checked endpoints	Adopt refactors with unchanged public statements
CM215	Very short exact relative machine proof	Final project endpoint composition/replay
ZC215	Direct all-prefix credit argument	Literal parser/loop, startup, and binary endpoint integration
FM215	Small four-row local transformation	Repository representation adapter and endpoint composition
GC-ZC215	Combined encoding research	Extra generalization and decoder obligations

7 A complementary, reproducible search for 214 states

7.1 Predecessor-specific elimination

Previous searches explored both fixed-pair merges and cutpoint rewrites. In particular, the newest incoming proposal contains a two-row cutpoint search for helper 176, so deleting a state without one uniform state-map image is not a new idea. The present contribution systematically applies per-incoming-row choices to every eligible deletion target in 21 released tables, together with a

separate state-40 provider experiment. Its composability restrictions differ from the earlier cutpoint search; neither grammar contains the other in general.

Choose a noninitial control d to delete. For every external transition row that enters d, independently choose a replacement write, direction, and retained destination or halt. Different predecessors may route to different continuations. Thus successful elimination need not identify d with one common surviving state.

For each proposed replacement row and each admitted local tape window, the checker follows the old and edited computations until they meet at exactly the same surviving control, head displacement, and full local tape contents. Neither path may leave the tracked window before the certified meeting. Thus all cells outside that window are framed unchanged. Nontrivial blocks have positive duration. Other simultaneously changed rows are excluded from the macro interior; repeated uses of the currently checked row are handled with the proposed replacement. These restrictions make independently accepted incoming-row implementations composable.

The input windows are computed by a finite abstract reachability closure. The implemented closure starts from every local window at the declared start control; when the head moves, an unobserved boundary bit is conservatively allowed either value. Thus it overapproximates starts on arbitrary tapes, including blank input. This stronger invariant supports sound positive certificates, but can reject a valid blank-only optimization because an admitted window is actually unreachable from the blank configuration.

7.2 Completed scope and outcome

The broad run tested 7,324,620 local replacement actions across 21 released tables and performed 8,475,730 guarded case tests. It found no complete state elimination. A targeted provider search tested 731,000 provider actions, retained 1,812 by its preservation criterion, and then tested 1,558,320 root actions without a successful repair.

The principal run uses radius five and a 96-step local trace cap on 21 released 215-state tables. A second run tests the primary at radius seven and a larger 192-step cap. There is also a targeted search that changes a provider row while trying to bypass state 40. Positive controls and exact input hashes accompany the code and receipts.

The 21 inputs have distinct rooted transition graphs, and remain 21 distinct classes when global reflection is allowed. All 215 controls are graph-reachable in each table. The dominant rejection category is leaving the finite window (6,337,648 local trials). These counts describe failed sufficient certificates, not semantic counterexamples to whole machines. The local-join transformation theorem in the search README is an ordinary mathematical argument; it is not advertised as a new kernel-checked theorem.

The quantities called action trials are local row proposals, not numbers of distinct complete Turing machines. The broad run does not enumerate all combinations of successful incoming-row choices when one required predecessor has no accepted implementation. This early rejection is sufficient for the specific independently composable grammar but says nothing about mutually dependent coordinated rewrites outside it.

7.3 What the closest obstruction teaches

On the primary table, state 40 provides a useful near miss: two of its three external incoming rows can be rerouted, while the third remains unresolved. States 40 and 54 jointly scan left for the first double zero. The two roles remember whether a single zero has just been seen. That explains why treating them as interchangeable decoder continuations loses necessary information.

This observation is not a general lower bound for a one-tape Turing machine. A different encoding, a temporary tape mark, a stronger reachable-state invariant, or a simulation relation with shifted comparison points might store that information elsewhere. The failed provider search records one finite attempt to do so with existing rows; it does not rule out those broader possibilities.

An independent dictionary-tape replayer reduces the failure at predecessor `54/0` to three admitted radius-five windows: `00001010000`, `00101010000`, and `00000010100`, with the head on the sixth bit. Of 860 possible replacement actions, three pass the first window, two pass the first two, and none passes all three under the specified bounds. These are abstract-invariant contexts; their occurrence on the blank run is not claimed. A separate positive control deletes a live shared state using two different providers and is independently replayed on all 64 nonvacuous contexts.

The search treats fuel exhaustion, leaving the tracked window, and failure to find a bounded exact join as rejection by the template. They are not all counterexamples to the proposed whole machine. No claim of minimality follows from a large action-trial count.

8 Integration plan and further research

8.1 Recommended incorporation sequence

1. **Adopt the normalized Rocq source layer.** Preserve the literal source programs and existing theorem interface. Add the reusable checker and replace the large offset-stage source proof by its 1,426-entry one-sided instance. Replay under the maintained Rocq toolchain and audit the full endpoint.
2. **Choose how to maintain Lean correspondence data.** The incoming normalized proof is already a good intermediate state. The automatic version generates its materialized certificate from the source trees. Keep that staged architecture and measure its cost against the existing materialized version before choosing the production default.
3. **Add CM215 through the existing exact transfer.** Reuse the common K215 calibration and complete the final checked project composition. Avoid duplicating source or bootstrap proofs in each small table variant.
4. **Integrate the concrete ZC215 codec and credit theorem.** Connect the implementation's command/refill semantics to the exact parser, then use the protected-head corollary to discharge stack safety. Keep the arbitrary tail as an explicit logical witness.
5. **Continue state synthesis with proof-producing interfaces.** A future successful 214 table should come with a literal state count, exact initial configuration, a total runtime relation, and positive progress or a well-founded stuttering measure. Bounded agreement alone is insufficient.

8.2 A concrete next attempt at 214 states

The most actionable new synthesis direction is to weaken the local physical register-file contract. The existing `Runtime230Core.v` increment box fixes its output to an exact file of the form `sepsR (incr r vs ++ [1])`. Its precondition supplies at least ten file entries while the simulated program accesses only the first nine logical registers. At the higher layer, `Sim230.v` already hides the spare extension existentially. Thus the proposed change is to the stronger local box specification, not the invention of a new global abstraction.[7]

Preserve the first nine registers and the logical continuation, but permit any nonempty valid spare suffix and an allowed gap on return:

$$\mathrm{enc}(p, \mathbf{r}, g, s) \;\longrightarrow^{+}\; \mathrm{enc}(p', \mathbf{r}', g', s'), \qquad s' \neq [].$$

Then resynthesize the register excursion, spare append, and return-to-dispatch macro jointly. This permits changes to old spare values and the number of spare blocks, beyond choosing whether a newly appended block contains zero or one. Exact whole-tape joins exclude these changes before their abstract semantic validity can be considered.

The new macro would still need a universal proof preserving the nine logical registers, well-formed nonempty suffix, permitted gap, every shared decoder use, positive progress, and exact blank startup. No machine realizing a saving under this relaxed contract has yet been constructed. The search packet gives the precise existing formal-source locations behind this proposal.

There is also a useful dependency cleanup. The file `Prog215.v` imports `K230.Prog230` for its encoded edge constructors and `cv`, pulling in an unrelated 5,124-node source table and well-formedness certificate. Extract those shared constructors into a small generic module imported by both programs. This simplification is proposed; its runtime effect has not been measured.

8.3 Specific further questions

1. Can automatic correspondence inference be specialized to verified annotated source DAGs, so repeated subtrees share checkpoint counts and segment summaries during kernel reduction? This would address computation cost while retaining the no-external-correspondence interface.

2. Can the normalization-aware relation be generalized from trailing-zero aliases to equality of the complete consumed primitive observation? That larger quotient can merge pointers with unused suffixes, but it requires careful successor and halt preservation.

3. Can the current direct halting-witness transport in the Rocq checker be factored into a generic segment relation theorem matching Lean's `Bisim` interface? The present implementation prioritizes a small, checked port over a broad library refactor.

4. Can the literal ZC215 register parser be related to the concrete codec by a short symbolic block proof, so the new credit theorem removes the older debt-accounting argument from an entire candidate endpoint?

5. Does a table with a slightly larger source but a simpler bootstrap admit a smaller total binary allocation? Source-DAG size, occurrence count, and working-state count should be optimized jointly, not used interchangeably.

6. Can the double-zero scanner's memory be moved into a temporary local marker that is restored at a different comparison boundary, eliminating a working control without adding a provider? The present exact local joins do not settle that representation change.

7. Can a reachable-phase certificate permit additional junk formula or stack configurations while preserving both directions of the consistency calibration? Equiconsistency needs soundness and completeness, even when the intermediate syntax has no intended set-theoretic interpretation.

8. After the source and local machine transport layers are compact, what is the smallest additional formal development needed to obtain an internally encoded PA or ZFC proof for the exact 215-state table? This remains distinct from the external proof-assistant theorem.

9 Artifact and verification guide

The archive is designed as an additive research packet. It does not modify or replace the repository checkout used for comparison. Its main components are `lean_auto/`, `rocq_normalized/`, `rocq_auto/`, `zc_credit/`, `search214/`, `evidence/`, this article, and its editable source. Each component's README gives a portable rebuild command and states the boundary between its new theorem and inherited project dependencies.

The top-level manifest binds the delivered files. Hash checks establish artifact identity; they do not replace kernel checking. The packet retains compiler and kernel logs for the newly checked theorems and contains the source needed to reproduce those checks. It does not distribute toolchain binaries or compiled proof objects. Search receipts record their input tables, abstraction bounds, and unsuccessful outcomes.

The report's new formal claims are restricted to the recorded source and scheduler theorems. The original machine-to-ZFC theorem, the prior CM215 bridges, the shared-tree representation, and the normalized Lean checker are explicit inherited dependencies. No absolute consistency assertion, 214-state success, or universal state lower bound is claimed.

References

[1] ProveIt, *K215 formal verification record*, commit `53269bcab1ef`. Pinned verification README.

[2] ProveIt, *Shared implication lemmas and symbolic execution*, phase-four simplification record. Pinned phase-four README.

[3] ProveIt, seven pending research archives in `docs/incoming`, inspected at the pinned commit. Pinned incoming directory. Exact archive names and SHA-256 digests accompany this packet.

[4] ProveIt, *CM215: conditional origin markers and ZFC consistency*. Pinned CM215 construction and ordinary proof. Formal follow-up status is in the incoming archives.

[5] ProveIt, *ZC215: zero-credit enumeration*, including `proofs/tag_lowering_credit.md`. Pinned ZC215 README.

[6] ProveIt, *RegisterPath, RMEquiv, and RMBisim*, maintained Lean source semantics. Pinned Lean module directory. The files preserve attribution to Wade's compiler framework and the Mira-Cult/Jeff verification on which their semantics are based.

[7] ProveIt, *Runtime230Core and Sim230*, selected runtime contracts for K215. Pinned local runtime contracts. The existential representation is in the adjacent `Sim230.v`.